

Neural Networks as Parameterizers of Probability Distributions — Solutions

ELG 5218 – Uncertainty Evaluation in Engineering Measurements and Machine Learning

Instructor: Miodrag Bolić, University of Ottawa

February 25, 2026

Part A: Conceptual Questions

A1. Universal Approximation and Distribution Parameterization

Question. Explain why a feedforward neural network can serve as a parameterizer of a probability distribution...

A feedforward neural network defines a parametric family of functions $f_\theta : \mathbb{R}^d \rightarrow \mathbb{R}^m$. By the universal approximation theorem [2, 4], with sufficient capacity, such a network can approximate any continuous function on a compact domain to arbitrary accuracy. Because a probability distribution is fully determined by a finite set of parameter functions (e.g., mean and variance for a Gaussian, or mixing weights and component parameters for a mixture), the network can learn to produce those parameters as a function of input x .

The additional step required is a **constraint mapping** $g(\cdot)$: raw network outputs are passed through appropriate transformations (softmax for probabilities, exp or softplus for variances, Cholesky decomposition for covariances) to enforce the mathematical constraints that make outputs valid distribution parameters. The composed map $\eta(x) = g(f_\theta(x))$ then outputs valid distribution parameters for any input.

A2. Constraint Mappings for Valid Outputs

- (a) **Mixture weights** π_k : apply **softmax** to the K raw output logits. Softmax ensures $\pi_k > 0$ for all k and $\sum_k \pi_k = 1$, satisfying the requirements of a discrete probability distribution.
- (b) **Variance** $\sigma^2(x)$: apply $\exp(\cdot)$ or $\text{softplus}(\cdot) = \log(1 + e^z)$ to the raw scalar output. Both guarantee a strictly positive value. Softplus is preferred when gradients near zero matter, as it avoids the saturation issues of exp for large negative inputs.
- (c) **Full covariance matrix** $\Sigma(x)$: parameterize via a lower-triangular Cholesky factor $L(x)$ with positive diagonal entries (diagonal entries pass through exp). Then $\Sigma = LL^\top$ is symmetric positive definite by construction. The network outputs the $d(d+1)/2$ free

entries of L .

A3. Conditional vs. Generative Modeling

Conditional density estimation targets $p_\theta(y \mid x)$. The network takes x as input and outputs parameters $\eta(x)$ of the conditional distribution; a sample y is drawn directly from $p(y; \eta(x))$. Example: predicting temperature from sensor voltage.

Generative modeling targets the marginal $p_\theta(x)$. A latent noise variable $z \sim p(z)$ (e.g., a standard Gaussian) is drawn and passed through a deterministic network f_θ to produce a sample $x = f_\theta(z)$. The network does not output $p_\theta(x)$ explicitly; rather, $p_\theta(x)$ is the distribution *induced on* x when z is random. Sampling proceeds by first drawing $z \sim p(z)$, then computing $x = f_\theta(z)$.

A4. Mixture Density Networks (MDNs)

For a K -component Gaussian MDN, the network must output for each component $k = 1, \dots, K$:

- **Mixing weight** $\pi_k(x)$: must satisfy $\pi_k > 0$ and $\sum_k \pi_k = 1$. Enforced by softmax over K raw logits.
- **Component mean** $\mu_k(x)$: unconstrained, output directly.
- **Component std** $\sigma_k(x)$: must be positive. Enforced by exp or softplus.

Total output dimensionality: $3K$ scalars for univariate y .

MDNs are preferable to a single Gaussian head when $p(y \mid x)$ is multi-modal. A single Gaussian can only represent a unimodal distribution; if the true conditional has two modes, the Gaussian head will average them, placing probability mass in a region of low true density. An MDN assigns each mode to a separate component and correctly captures bimodal or multi-modal structure. [1]

A5. Autoregressive Factorization

The probability chain rule states:

$$p(x_{1:D}) = \prod_{d=1}^D p(x_d \mid x_{<d}).$$

A neural AR(2) model applies the order-2 Markov approximation: each x_t depends only on (x_{t-2}, x_{t-1}) . Two small networks take (x_{t-2}, x_{t-1}) as input and output μ_θ and $\log \sigma_\theta$, defining

$$p(x_t \mid x_{t-1}, x_{t-2}) = \mathcal{N}(\mu_\theta(x_{t-1}, x_{t-2}), \sigma_\theta(x_{t-1}, x_{t-2})^2).$$

The advantage over linear AR(2) is that the neural network can capture **nonlinear dynam-**

ics (e.g., threshold effects, regime switches) and **heteroscedastic noise** where the variance depends on past states — both impossible with the linear form $x_t = a_1 x_{t-1} + a_2 x_{t-2} + \varepsilon$. [3]

A6. Latent-Variable Models and VAEs

The **decoder** $p_\theta(x | z)$ is a neural network that takes a latent vector z and outputs parameters of a distribution over observations x . It defines the generative story: $z \sim p(z)$, $x | z \sim p_\theta(x | z)$.

The **encoder** $q_\phi(z | x)$ is a neural network that takes an observation x and outputs parameters of a distribution over latent codes z (mean and variance of a Gaussian), approximating the true posterior $p_\theta(z | x)$.

The true posterior is **intractable** because computing it requires the marginal likelihood:

$$p_\theta(x) = \int p_\theta(x | z) p(z) dz,$$

which has no closed form for general neural decoders. The VAE instead maximizes the **Evidence Lower Bound (ELBO)**:

$$\mathcal{L}_{\text{ELBO}} = \mathbb{E}_{q_\phi}[\log p_\theta(x | z)] - \text{KL}(q_\phi(z | x) \| p(z)),$$

which is tractable and equals $\log p_\theta(x)$ minus the non-negative KL gap. [6]

A7. Learning Variance Without Variance Labels

The Gaussian NLL loss is:

$$\mathcal{L}(x, y) = \log \sigma_\theta(x) + \frac{(y - \mu_\theta(x))^2}{2\sigma_\theta(x)^2}.$$

This loss penalizes two opposing behaviors:

- The $\log \sigma$ term penalizes **predicting large uncertainty**: a model that always outputs $\sigma \rightarrow \infty$ would have $\log \sigma \rightarrow \infty$, making the loss unbounded.
- The scaled residual term penalizes **predicting small uncertainty when residuals are large**: small σ makes large residuals $(y - \mu)^2 / \sigma^2$ extremely costly.

The optimizer therefore finds the $\sigma(x)$ that balances these two penalties: it predicts high σ where the mean is often wrong and low σ where predictions are accurate. No explicit σ labels are needed; the data scatter provides the signal. [8]

A8. Pathology: Variance Absorbing Mean Error

During joint training, a randomly initialized mean head $\mu_\theta(x)$ produces large residuals in certain input regions. The variance head can exploit this by increasing $\sigma_\theta(x)$ in those regions, which

reduces the scaled residual term $(y - \mu)^2/(2\sigma^2)$ in the NLL — lowering the loss even though the mean is wrong.

As shown in B3, the gradient for mean learning is $\partial\mathcal{L}/\partial\mu = -(y - \mu)/\sigma^2$, which is suppressed by $1/\sigma^2$. As σ grows large, the mean gradient effectively vanishes and **mean learning stalls**. The result is a model with inflated, spuriously structured uncertainty and a poorly trained mean.

The two recommended remedies are:

1. **Mean warm-up** [9]: train the mean head first with a fixed variance until convergence, so the mean is reasonably accurate before variance training begins.
2. **Variance regularization**: add a penalty on $\|\log \sigma_\theta(x)\|^2$ or $\|\nabla_x \sigma_\theta(x)\|^2$ to prevent the variance from growing arbitrarily large or becoming highly structured without data support.

Part B: Mathematical Derivations

B1. Gaussian Negative Log-Likelihood

Problem. Starting from the Gaussian pdf, derive the per-sample NLL...

The Gaussian probability density is:

$$p(y | x) = \frac{1}{\sigma_\theta(x)\sqrt{2\pi}} \exp\left(-\frac{(y - \mu_\theta(x))^2}{2\sigma_\theta(x)^2}\right).$$

Taking the negative logarithm:

$$-\log p(y | x) = \log \sigma_\theta(x) + \frac{1}{2} \log(2\pi) + \frac{(y - \mu_\theta(x))^2}{2\sigma_\theta(x)^2}.$$

Dropping the constant $\frac{1}{2} \log(2\pi)$:

$$\mathcal{L}(x, y) = \log \sigma_\theta(x) + \frac{(y - \mu_\theta(x))^2}{2\sigma_\theta(x)^2}.$$

Interpretation:

- $\log \sigma_\theta(x)$: *complexity penalty* — discourages predicting arbitrarily large uncertainty.
- $(y - \mu_\theta(x))^2/(2\sigma_\theta(x)^2)$: *scaled squared residual* — penalizes poor mean predictions; the $1/\sigma^2$ scaling reduces the penalty when the model correctly expresses high uncertainty.

B2. Optimal Variance at a Fixed Input

With $r = y - \mu^*$ fixed, the NLL as a function of $\sigma > 0$ alone is:

$$\ell(\sigma) = \log \sigma + \frac{r^2}{2\sigma^2}.$$

Differentiating and setting to zero:

$$\frac{d\ell}{d\sigma} = \frac{1}{\sigma} - \frac{r^2}{\sigma^3} = 0 \implies \sigma^2 - r^2 = 0 \implies \boxed{\sigma^2 = r^2}.$$

The second derivative $d^2\ell/d\sigma^2 = -1/\sigma^2 + 3r^2/\sigma^4$, evaluated at the stationary point $\sigma^2 = r^2$, gives $2/\sigma^2 > 0$, confirming a minimum.

Interpretation over a distribution of residuals: over many samples at similar x , the minimizer satisfies

$$\sigma(x)^2 \approx \mathbb{E}[(y - \mu(x))^2 \mid x],$$

which is precisely the conditional variance. No explicit variance labels are needed; the data scatter provides the supervisory signal. [8]

B3. Gradient of the NLL with Respect to μ_θ and $\log \sigma_\theta$

Let $s = \log \sigma_\theta(x)$, so $\sigma = e^s$. Rewrite the NLL:

$$\mathcal{L} = s + \frac{(y - \mu)^2}{2e^{2s}}.$$

Partial derivative w.r.t. μ :

$$\frac{\partial \mathcal{L}}{\partial \mu} = -\frac{y - \mu}{e^{2s}} = -\frac{y - \mu}{\sigma^2}.$$

Partial derivative w.r.t. s :

$$\frac{\partial \mathcal{L}}{\partial s} = 1 - \frac{(y - \mu)^2}{e^{2s}} = 1 - \frac{(y - \mu)^2}{\sigma^2}.$$

Key implication: the gradient for mean learning is scaled by $1/\sigma^2$. When $\sigma_\theta(x)$ is large, this gradient is very small — the mean receives almost no training signal. This is the mechanism underlying the pathology in A8: a network that inflates σ to cover poor mean predictions effectively turns off mean learning in that region. [5]

B4. AR(2) Training Objective

From the raw time series $x_1, \dots, x_T$, construct training triples:

$$\{(x_{t-2}, x_{t-1}), x_t\}, \quad t = 3, 4, \dots, T,$$

yielding $T - 2$ training examples. Per-sample loss:

$$\ell_t = \log \sigma_\theta(x_{t-1}, x_{t-2}) + \frac{(x_t - \mu_\theta(x_{t-1}, x_{t-2}))^2}{2 \sigma_\theta(x_{t-1}, x_{t-2})^2}.$$

Total training objective:

$$\mathcal{L}(\theta) = \sum_{t=3}^T \ell_t.$$

This is minimized by gradient descent. Note that the triples overlap (sliding window of size 2), so consecutive training examples share inputs. A **time-based train/test split** is required — not random shuffling — to avoid look-ahead bias.

B5. Calibration: Standardized Residuals

The standardized residual is defined as:

$$z_i = \frac{y_i - \mu_\theta(x_i)}{\sigma_\theta(x_i)}.$$

If the model is perfectly calibrated: $z_i \sim \mathcal{N}(0, 1)$ i.i.d.

Expected empirical coverage:

$$\Pr(|y - \mu| \leq 1\sigma) \approx 0.683, \quad \Pr(|y - \mu| \leq 2\sigma) \approx 0.954.$$

If **80%** of test points fall within $\pm 1\sigma$ (vs. the expected 68.3%), the model is **under-confident**: predicted σ values are systematically too large relative to actual errors. The intervals are wider than necessary, meaning the model is over-estimating uncertainty. This can result from excessive variance regularization, too-long warm-up, or a true noise level lower than what the model has learned. [7]

Part C: Parametric Analysis

C1. Effect of the Number of Mixture Components K

- (a) **Expressiveness**: increasing K allows the MDN to represent increasingly complex, multi-modal conditional densities. In the limit of large K , the mixture can approximate any smooth density (universal approximation for density estimation).
- (b) **Output units**: for univariate y , each component requires (π_k, μ_k, σ_k) , so the output layer has $3K$ units. For multivariate $y \in \mathbb{R}^d$, this scales with d (and d^2 per component if full covariances are used).
- (c) **Overfitting risk**: larger K means more output parameters. With limited data, the model may assign components to outliers or spurious modes. Cross-validation or held-out NLL should guide the choice of K .

In the limit $K = 1$, the MDN reduces exactly to a standard Gaussian-head (MVE) network.

C2. Effect of Variance Regularization Strength λ

As $\lambda \rightarrow 0$: the regularization has no effect. The variance head is free to take any value, including very large values in regions where the mean is imperfect. This enables the pathology of variance absorbing mean error (A8): $\sigma_\theta(x)$ can grow spuriously large to mask mean errors, producing overconfident intervals or meaningless uncertainty structure.

As $\lambda \rightarrow \infty$: the penalty drives $\log \sigma_\theta \rightarrow 0$, hence $\sigma_\theta \rightarrow 1$ everywhere. The model produces **homoscedastic** uncertainty regardless of actual data scatter, underfitting the true variance. Predictions have uniform uncertainty even where data clearly supports heteroscedasticity.

The practical choice of λ should be treated as a hyperparameter tuned on a validation set using held-out NLL or calibration metrics.

C3. Warm-Up Duration

(a) Too short warm-up ($T_{\text{warm}} \approx 0$): the mean head is randomly initialized and produces large residuals. The variance head immediately inflates $\sigma_\theta(x)$ to cover these errors. As shown in B3, the gradient for μ_θ is then suppressed by $1/\sigma^2$, trapping the model in a poor solution where σ is large and the mean never improves.

(b) Too long warm-up: the mean head has converged under fixed variance (MSE objective). When the variance head is unfrozen, the residuals are stable and provide a clean signal for $\sigma(x)$. The risk is that the mean converged to a solution optimal for MSE but not necessarily for NLL, and subsequent joint training may not readjust it significantly. Moderate warm-up is sufficient.

Role of warm-up in summary: warm-up stabilizes mean learning first, so that variance training receives meaningful, stable residuals as its training signal and mean gradients are not suppressed by early high-variance predictions. [9]

C4. Heteroscedasticity vs. Homoscedasticity

Ideally, the network should learn $\sigma_\theta(x) \approx \sigma$ for all x — a flat function. Several factors can cause a non-constant $\sigma_\theta(x)$ even when the true noise is homoscedastic:

- **Mean model error:** if $\mu_\theta(x)$ is imperfect in some regions (higher bias), residuals vary spatially. The variance head learns to track these spatially varying residuals even though the true noise is constant. This is the pathology from A8.
- **Sparse data regions:** with few observations, empirical variance estimates are noisy and the network may overfit finite-sample fluctuations.
- **Excess capacity in the variance head:** an overly expressive variance network can fit non-constant patterns in the residuals that are purely due to sampling noise. Regulariza-

tion of $\sigma_\theta(x)$ is the standard remedy.

Part D: Application and Notebook Questions

D1. Engineering Application – Probabilistic Load Forecasting

- (a) A standard regression model outputs a single number $\hat{y}$ with no accompanying uncertainty. Grid operators need *prediction intervals* to provision reserves and plan contingencies. A point estimate cannot represent the range of plausible demand values. Furthermore, bimodal load distributions cannot be represented by a single number.
- (b) An **MDN** is more appropriate. The problem states that the load distribution is bimodal on weekends. A single Gaussian head would average the two modes, placing high probability in a low-density intermediate region. An MDN with $K \geq 2$ components can represent both modes simultaneously, with mixing weights $\pi_k(x)$ that adapt to day type (weekday vs. weekend) based on the input features.
- (c) Without explicit uncertainty labels, calibration is assessed via: (i) **standardized residuals** $z_i = (y_i - \hat{\mu}_i)/\hat{\sigma}_i$ — should follow $\mathcal{N}(0,1)$ for a Gaussian head; (ii) **empirical coverage** — check that $\approx 68\%$ of held-out observations fall within $\pm 1\hat{\sigma}$ and $\approx 95\%$ within $\pm 2\hat{\sigma}$; (iii) **held-out NLL** as a proper scoring rule.
- (d) A well-calibrated model should express higher uncertainty at OOD inputs — wider prediction intervals reflecting extrapolation. The fundamental limitation of a standard Gaussian-head network is that $\sigma_\theta(x)$ is a deterministic function trained on in-distribution data, with no guarantee it increases for OOD x . Neural networks can be *overconfident* on OOD inputs. Deep ensembles or Bayesian NNs are better suited for OOD uncertainty, as they capture epistemic uncertainty from limited training coverage. [7]

D2. Notebook: AR(2) Neural Network

- (a) If the true noise is homoscedastic, $\sigma_\theta(x_{t-1}, x_{t-2})$ should converge to a roughly **flat, constant** value close to the true σ . Patterns indicating overfitting include: σ_θ following the amplitude or frequency of the time series (rising during large oscillations, falling during quiet periods), or rapid oscillations with no temporal structure — both suggest the variance head is fitting noise or mean model errors rather than true aleatoric uncertainty.
- (b) With two-phase training, the mean head first converges on an MSE objective, establishing reasonable μ_θ estimates. The variance head then trains with stable residuals. Expected observations: **lower final NLL** than joint training (less variance-absorbs-mean risk), **better calibration** (empirical coverage closer to 68%/95%), and less erratic $\sigma_\theta(t)$.
- (c) With σ_θ inflated by $3\times$, generated sequences have much higher per-step variance. In autoregressive generation, errors compound: each sample x_t serves as input to the next step, so large noise propagates forward. The result is that generated sequences **quickly diverge** from plausible trajectories — exhibiting unrealistically large amplitude swings and failing to maintain the temporal structure of the true AR(2) process. This highlights

that well-calibrated uncertainty is critical in generative models.

(d) Changes required for a 2-component Gaussian MDN AR(2):

- **Architecture:** add a third output head — two raw mixing logits (or a single logit whose sigmoid gives π_1 , $\pi_2 = 1 - \pi_1$), two mean outputs (μ_1, μ_2) , and two log-std outputs $(\log \sigma_1, \log \sigma_2)$.
- **Output layer:** apply softmax to logits $\rightarrow (\pi_1, \pi_2)$, exp to log-stds.
- **Loss function:** replace Gaussian NLL with the MDN NLL:

$$\ell_t = -\log[\pi_1 \mathcal{N}(x_t; \mu_1, \sigma_1^2) + \pi_2 \mathcal{N}(x_t; \mu_2, \sigma_2^2)].$$

- **Sampling:** draw component $k \sim \text{Categorical}(\pi_1, \pi_2)$, then $x_t \sim \mathcal{N}(\mu_k, \sigma_k^2)$.

D3. Comparing Distribution Families

- (a) **Gaussian head (MVE).** The output is scalar and the uncertainty is unimodal but heteroscedastic (growing variance as the battery ages). An MVE network with input-dependent $\sigma(x)$ is the natural choice; the growing variance with degradation is exactly the kind of aleatoric uncertainty these networks capture.
- (b) **Autoregressive model** (e.g., PixelCNN / MADE). The joint distribution of 784 pixels is high-dimensional with strong spatial dependencies. Autoregressive factorization $p(x_1, \dots, x_{784}) = \prod_i p(x_i | x_{<i})$ provides a tractable exact likelihood and is well-suited for high-dimensional density estimation.
- (c) **MDN.** The mapping from end-effector position to joint angles is one-to-many: for a given target position, multiple geometrically valid arm configurations exist. A single Gaussian head would average these solutions, producing an invalid intermediate configuration. An MDN with K components can represent each valid solution branch as a separate mixture component.
- (d) **VAE.** The VAE encoder learns a structured latent space where normal data is compressed to a compact region. Anomalous inputs produce latent codes with high reconstruction error or low prior probability — both serve as anomaly scores. The VAE is designed for learning useful representations of the data distribution, making it ideal for this task.

References

- [1] Bishop, C.M. (1994). *Mixture Density Networks*. Neural Computing Research Group Technical Report NCRG/94/004.
- [2] Cybenko, G. (1989). Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals and Systems*, 2(4), 303–314.
- [3] Germain, M. et al. (2015). MADE: Masked autoencoder for distribution estimation. *ICML*.

- [4] Hornik, K. (1991). Approximation capabilities of multilayer feedforward networks. *Neural Networks*, 4(2), 251–257.
- [5] Immer, A. et al. (2023). Effective Bayesian heteroscedastic regression with deep neural networks. *NeurIPS*.
- [6] Kingma, D.P. & Welling, M. (2013). Auto-encoding variational Bayes. *arXiv:1312.6114*.
- [7] Lakshminarayanan, B. et al. (2017). Simple and scalable predictive uncertainty estimation using deep ensembles. *NeurIPS*.
- [8] Nix, D.A. & Weigend, A.S. (1994). Estimating the mean and variance of the target probability distribution. *IEEE ICNN*.
- [9] Sluijterman, L. et al. (2023). Optimal training of mean variance estimation neural networks. *arXiv:2302.08875*.

Acknowledgements / Document Preparation

The problems and solutions in this document were generated with assistance from large language models (ChatGPT by OpenAI, CoPilot by Microsoft, and Claude by Anthropic), based on the course slide and notebook material. The instructor reviewed, modified, and verified all questions, solutions, and final formatting.